Prophet Hacks Forecasting System Dossier

Research-grade plan, monitoring architecture, and implementation guide

Rob Sneiderman

2026-05-16

Contents

Opening: what this artifact is **2**

Quick answers to the current strategic questions **3**
- Are we using frontier models? . . . 3
- Are we using batch jobs? . . . 3
- Should we set up a sandbox for agents? . . . 4
- Should we set up a RAG / message system? . . . 4
- Should we scrape Discord, Kalshi, Polymarket, or other sources? . . . 5

Definitions and terms **5**
- Forecasting . . . 5
- Event, market, and outcome . . . 5
- Market prior . . . 5
- Model probability . . . 5
- Final probability . . . 6
- Edge . . . 6
- Brier score . . . 6
- ECE / calibration error . . . 6
- Market return . . . 6
- RAG / retrieval . . . 6
- Stacking and ensembling . . . 6
- Small area estimation inspiration . . . 7

Research foundation: what the papers tell us **7**
- Prophet Arena / LLM-as-a-Prophet . . . 7
- Kalshi market microstructure paper . . . 7

System architecture **8**
- Proposed module layout . . . 8

Frontier model routing **9**
- Default forecast route . . . 9
- Reasoning effort policy . . . 9

Batch jobs and API-credit plan **10**
- Offline batch experiments . . . 10
- Credit allocation . . . 10

Small area estimation as a real edge **10**
- SAE analogy . . . 10
- Why this helps . . . 11
- Practical model . . . 11
- Credibility weighting version . . . 11

Data acquisition: APIs, scraping, and useful signals **12**
Source classes . . . 12
Scraper design if we use it . . . 12

Monitoring and visualization **13**
Daily dashboard . . . 13
Outcome dashboard . . . 13
Status file . . . 14

Tests and smoke ladder **14**
Smoke ladder . . . 14
Unit tests to add . . . 14
Paper-derived tests . . . 14

Hyperparameter search and stacking **15**
Hyperparameters worth tuning . . . 15
Objective hierarchy . . . 15
Time-based validation . . . 15

Toy example **15**

More realistic example: low-price longshot **16**

Research artifact plan **16**

Immediate implementation roadmap **16**
Patch 1: monitoring and status . . . 16
Patch 2: frontier model forecast path . . . 17
Patch 3: calibrated blend . . . 17
Patch 4: CI and artifacts . . . 17
Patch 5: optional connectors . . . 17

Open questions to fill in **17**

References and source anchors **17**

Closing direction **18**

Opening: what this artifact is

This report is prepared for **Rob Sneiderman** as the operating and research dossier for Prophet Hacks. It is meant to be useful in three ways:

1. **Competition plan:** define the system we are going to build, monitor, and submit.
2. **Research artifact:** preserve the ideas, evidence, and experimental decisions well enough that the project can become a serious post-event write-up.
3. **Agent context:** give coding agents, evaluation agents, and future collaborators enough context to avoid relitigating the same decisions.

The goal is not to build a cute chatbot that guesses outcomes. The goal is to build a **forecasting system**: a monitored, reproducible, probability-producing machine that uses market data, frontier models, retrieval, calibration, borrowing of information, risk gates, and trace logs.

Working thesis: The best contest system is likely a market-aware, frontier-model-assisted, small-area-estimation-inspired calibrated ensemble. The LLMs are not the whole system. They are one class of noisy forecasters whose estimates must be pooled, shrunken, audited, and tested against market baselines.

Blunt principle: If a component cannot improve Brier score, calibration, market-relative edge, monitoring clarity, or submission reliability, it is not part of the critical path.

Figure 1: System architecture

Quick answers to the current strategic questions

Are we using frontier models?

Yes. The default serious forecasting stack should use frontier models. The phrase "cheap model" should not mean "weak default forecaster." It means **cheap tasks get cheap models**.

Use frontier models for:

- final probability estimates on eligible markets;
- contradictory evidence synthesis;
- near-threshold trade review;
- hard domains such as politics, macro, geopolitics, public health, science, finance, and ambiguous resolution rules;
- model-averaging experiments where independent model errors matter.

Use cheaper or smaller models for:

- domain classification;
- source deduplication;
- source summarization;
- simple parse/format validation;
- obvious skip decisions;
- offline cheap ablations.

A good routing rule is:

```
All markets: market baseline + filters
Eligible markets: GPT-5.5 or equivalent frontier primary forecast
Near-threshold/conflict markets: Opus / second frontier review
Simple monitoring/classification: cheap model or no model
```

OpenAI's GPT-5.5 docs describe it as strong for tool-heavy agents, grounded assistants, long-context retrieval, and complex production workflows, and recommend treating reasoning effort as a tunable knob rather than always increasing it. Anthropic's Claude docs describe Opus 4.7 as the most capable generally available Claude model for complex reasoning and agentic coding. These are exactly the kinds of models we use for synthesis and review, not for every tiny bookkeeping step.

Are we using batch jobs?

Yes, for **offline experiments**. No, for live tick decisions.

Batch jobs are ideal for:

- running 200-2000 pastcast examples;
- comparing prompt variants;
- generating model forecasts for stacker/calibrator training;

- source-quality audits;
- LLM-as-judge audits;
- embeddings for project RAG.

Batch jobs are wrong for:

- live 15-minute tick decisions;
- near-resolution markets;
- anything that needs immediate response;
- final trade execution.

The practical split:

```
Offline research: Batch API, cheaper, asynchronous, large N.
Live agent: synchronous API, small N, strict budget and latency cap.
```

Should we set up a sandbox for agents?

Yes. But the sandbox should be boring and controlled:

```
repo/
  agent.py                        # submitted path
  experiments/                    # sandbox variants, not default submission
  examples/                       # toy events and mock traces
  tools/                          # monitor/eval/compliance utilities
  reports/status.md               # current project state
  reports/runs/                   # run summaries
  trace/                          # ignored local traces
  .github/workflows/verify.yml    # CI gates
```

Agents can create branches, patches, toy modules, and experiment files. The default submitted path should remain simple. No agent gets to silently change the production strategy without a traceable diff and a run summary.

Should we set up a RAG / message system?

Yes, but as a **project control plane**, not as another autonomous trading agent.

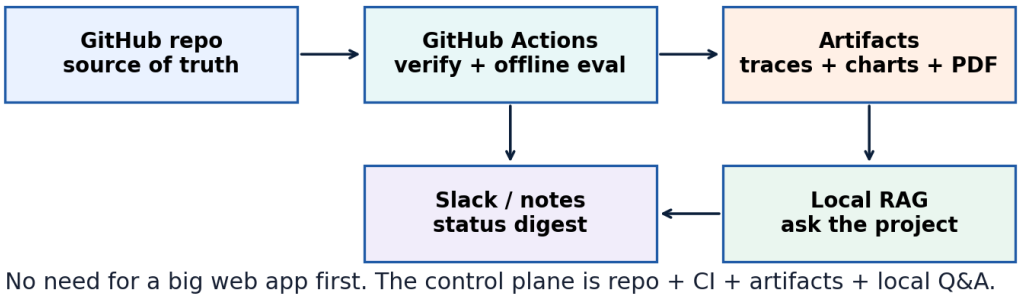

Figure 2: Control plane

The first version should be:

- GitHub repo as source of truth;
- GitHub Actions for verification and offline eval;
- artifacts uploaded from CI: traces, charts, summaries;
- a local project RAG to ask questions about docs, papers, run logs, and decisions;
- optional Slack webhook for run summaries and alerts.

Do not build a full interactive Slack bot before the core forecasting loop is stable.

Should we scrape Discord, Kalshi, Polymarket, or other sources?

The useful answer is not "never scrape." The useful answer is **value first, dependency second**.

We can collect external signals if they help. But the agent should not become dependent on brittle scraping. Priority order:

1. benchmark payload and official contest API;
2. structured APIs and public endpoints;
3. targeted web retrieval;
4. small purpose-built scrapers if a source is genuinely useful and there is no better structured route;
5. manual notes for organizer messages and teammate decisions.

For Discord specifically, the immediate value is likely operational: announcements, rule clarifications, status notes. That does not require bulk scraping. For Kalshi and Polymarket, official APIs or public market pages are more valuable than generic scraping because they produce structured prices and order-book context.

Implementation stance: build `data_sources/` connectors behind feature flags. Let them enrich forecasts when useful. Do not let any connector become required for the submitted agent to run.

Definitions and terms

This section is intentionally basic. It exists so every agent working on the project shares the same vocabulary.

Forecasting

A **forecast** is a probability assigned before an outcome is known. In this project most forecasts are binary: an event either resolves **YES** or **NO**.

A good forecast is not a dramatic prediction. A good forecast is a probability that is:

- calibrated: 70% forecasts happen about 70% of the time;
- discriminative: likely events receive higher probabilities than unlikely ones;
- useful: it improves a decision, trade, allocation, or prioritization;
- updateable: it changes when new evidence justifies a change;
- auditable: we can reconstruct why the probability was assigned.

Event, market, and outcome

An **event** is the broad question: for example, "Who will win the 2025 NBA Championship?"

A **market** is a specific binary proposition under that event: for example, "Will the Boston Celtics win?"

An **outcome** is the realized value after resolution:

```
o = 1 if YES resolves true
o = 0 if YES resolves false
```

The Prophet Arena paper formalizes this event/market structure and evaluates model probabilities at the binary market level, even when one event contains many markets.

Market prior

The **market prior** is the probability implied by current market prices. We denote it as `q_market` or `p_market`.

It is not sacred truth. It is a strong prior because market prices aggregate human beliefs, liquidity, and current information. It can still be biased, stale, or distorted by spread and fees.

Model probability

The **model probability** is the probability produced by a model, such as GPT-5.5 or Claude Opus:

```
p_model = model's estimate that YES resolves true
```

This is a noisy estimate. It should not directly determine trade size. It feeds a calibration and blending layer.

Final probability

The **final probability** is the system's decision probability after blending, shrinkage, guards, and calibration:

```
p_final = calibrated system probability
```

This is the number that should be logged, scored, and compared against the market.

Edge

For executable trading, edge is not "model confidence." Edge is the difference between our probability and the executable price.

```
YES edge = p_final - yes_ask
NO edge  = (1 - p_final) - no_ask
```

A trade should occur only when edge clears a threshold that accounts for spread, uncertainty, disagreement, and source quality.

Brier score

The **Brier score** measures squared probability error:

```
Brier = (p_final - outcome)^2
```

Lower is better. A random 50/50 forecast has expected Brier around 0.25 for balanced binary outcomes.

ECE / calibration error

Expected calibration error measures whether predicted probabilities match empirical frequencies across bins. If we predict 80% on 100 similar cases, about 80 should resolve YES.

Calibration matters because risk-sensitive decisions can prefer a well-calibrated forecaster even when raw Brier is not best.

Market return

Market return measures whether our forecast identifies mispricing relative to the market. Prophet Arena treats Brier, ECE, and market return as complementary metrics, not substitutes.

A forecast can have better Brier but worse return if it is on the wrong side of market mispricing. A forecast can have worse Brier but still identify profitable edge.

RAG / retrieval

Retrieval-augmented generation means fetching current external evidence and adding it to the model context. Retrieval should be targeted. More sources are not automatically better.

For our system, retrieval must log:

```
url, title, source type, retrieved_at, published_at, source_quality, supports_yes/no, staleness
```

Stacking and ensembling

Ensembling combines several forecasts. **Stacking** learns how to combine them from past performance.

Base forecasters might include:

- market baseline;
- GPT-5.5 no retrieval;
- GPT-5.5 with retrieval;
- bidirectional elicitation;
- Claude Opus review;
- domain rule forecaster.

The stacker should usually combine probabilities in logit space.

Small area estimation inspiration

Small area estimation is about borrowing strength across sparse groups. Here the "areas" are cells such as:

```
domain x horizon bucket x price bucket x source-quality bucket
```

Each cell may have few resolved examples, so we borrow information from related cells instead of overfitting each one separately.

Research foundation: what the papers tell us

Prophet Arena / LLM-as-a-Prophet

The Prophet Arena paper frames open-domain forecasting as a live, contamination-resistant benchmark for predictive intelligence. Its pipeline is exactly what we should mirror:

```
event/market extraction -> prediction context construction -> probabilistic prediction -> evaluation
```

Core lessons for our build:

- market baseline is a mandatory comparator;
- market data plus sources often performs best;
- market-only is already strong;
- sources can clarify or confound;
- models can be conservative relative to markets;
- markets can update faster near resolution;
- reasoning-to-probability alignment is a bottleneck;
- Brier, ECE, and market return answer different questions.

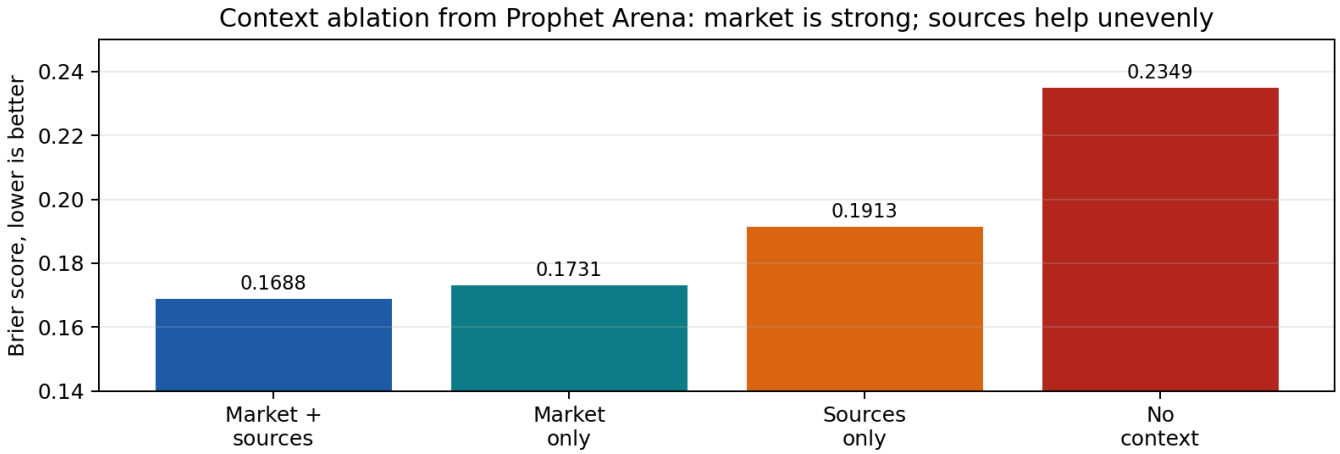

Figure 3: Context ablation

The paper reports a context ablation where market + sources is best, market-only is close, sources-only is worse, and no context is worst. The lesson is not "retrieve everything." The lesson is "use market context, add high-quality sources selectively, and monitor whether retrieval helped."

Kalshi market microstructure paper

The Kalshi paper gives us a market-microstructure warning: prices are informative, but not unbiased. The most important result for our system is the favorite-longshot bias.

Key findings:

- Kalshi prices generally predict outcomes and improve near closing.
- Low-price contracts win less often than prices imply.
- Contracts under 10c lose heavily on average.
- Higher-price contracts show small positive returns.

- Makers outperform Takers, but both show the longshot pattern.
- A modest probability-overweighting bias plus belief disagreement can reproduce the observed pattern.

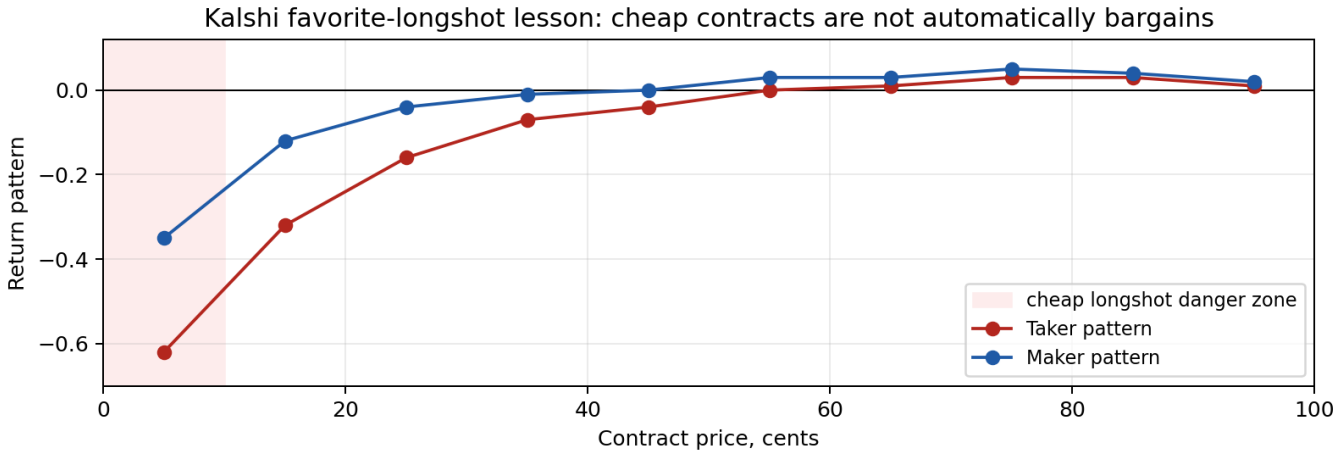

Figure 4: Kalshi longshot pattern

System consequences:

- do not let a vivid LLM story aggressively lift a 3c or 7c market;
- require stronger evidence for longshots;
- monitor performance by price bucket;
- near resolution, respect market updates more;
- use executable price and spread, not just midpoint;
- LLMs may reproduce the same human overweighting of small probabilities.

System architecture

The target system is a modular forecast-and-action pipeline.

```
1. Market snapshot
2. Eligibility filter
3. Domain/horizon/price-bucket routing
4. Evidence collection if useful
5. Frontier model forecast
6. Optional second frontier review
7. Ensemble / stacker
8. Small-area-style shrinkage / calibration
9. Longshot guard and risk gate
10. Action or skip
11. Trace logging
12. Offline/live evaluation
```

The LLM should not directly decide trade size. The LLM outputs structured probability, evidence interpretation, uncertainty, and rationale. Code computes calibrated probability, edge, size, and action.

Proposed module layout

```
forecasting/
  domain_router.py
  prompts.py
  providers_openai.py
  providers_anthropic.py
  retrieval_gate.py
  source_scoring.py
```

```
  bidirectional.py
  ensemble.py
  market_blend.py
  sae_shrinkage.py
  longshot_guard.py

trading/
  edge.py
  sizing.py
  risk.py

evaluation/
  brier.py
  ece.py
  reliability.py
  returns.py
  price_bucket_diagnostics.py

tools/
  monitor_run.py
  evaluate_offline.py
  hyperparam_search.py
  check_no_secrets.py
  check_compliance.py
  ask_project.py
```

Frontier model routing

We are not avoiding frontier models. We are avoiding stupid use of frontier models.

Default forecast route

```
Step 1: compute market baseline and filters without model call.
Step 2: route eligible markets to GPT-5.5 / frontier primary forecaster.
Step 3: if near threshold, conflict, or high-value domain, call Claude Opus / second frontier model.
Step 4: combine in logit space.
Step 5: shrink toward market based on evidence quality, disagreement, and horizon.
```

Reasoning effort policy

Use reasoning effort as a real hyperparameter:

Task	Model	Reasoning effort	Why
Parse/filter/format	none or cheap	none/low	deterministic or simple
Source scoring	GPT-5.5 or cheap	low	structured judgement, many calls
Primary forecast	GPT-5.5	medium	balanced quality/cost
Hard synthesis	GPT-5.5 / Opus	high only if eval shows gain	expensive and slower
Offline ablations	batch frontier	varied	estimate performance curves

The phrase “use frontier” does not mean “max reasoning on every market.” It means the critical probability-producing step uses the best available model, while everything else is routed for cost and latency.

Batch jobs and API-credit plan

Batch jobs are how we turn API credits into evidence instead of vibes.

Offline batch experiments

Run these before live:

1. Market-only baseline.
2. GPT-5.5 no retrieval.
3. GPT-5.5 with retrieval.
4. Bidirectional elicitation.
5. GPT-5.5 plus Opus review.
6. Calibrated market blend.
7. SAE-inspired shrinkage.
8. Stacked ensemble.

Each run logs:

```
variant, event_id, market_id, p_market, p_model, p_final, source_quality, model_disagreement, domain, horiz
```

Credit allocation

Use credits in phases:

Phase	Spend target	Use
0: zero/near-zero	$0-10	unit tests, toy examples, deterministic metrics
1: small API	$20-75	prompt schema validation, 50-100 examples
2: real offline batch	$100-300	500-2000 forecasts, stacker/calibrator search
3: live	bounded per tick	small number of frontier calls with strict routing

OpenAI Batch gives lower-cost asynchronous processing, higher rate-limit headroom, and a 24-hour completion window. That is excellent for offline evaluation and bad for live tick decisions.

Small area estimation as a real edge

This is worth including because it maps naturally to the problem and to Rob's statistical background.

SAE analogy

In classical small area estimation, each area has a noisy direct estimate and limited sample size. We borrow strength from related areas using a model.

For Prophet Hacks:

SAE concept	Forecasting analogue
area	domain x horizon x price bucket
direct estimator	market-implied probability or market baseline performance
synthetic estimator	model forecast using covariates/context
area covariates	domain, source quality, spread, volume, horizon, price bucket
random effect	domain/horizon-specific calibration residual
shrinkage	pull noisy cell estimates toward global pattern

SAE concept	Forecasting analogue
MSE	Brier, ECE, return variance

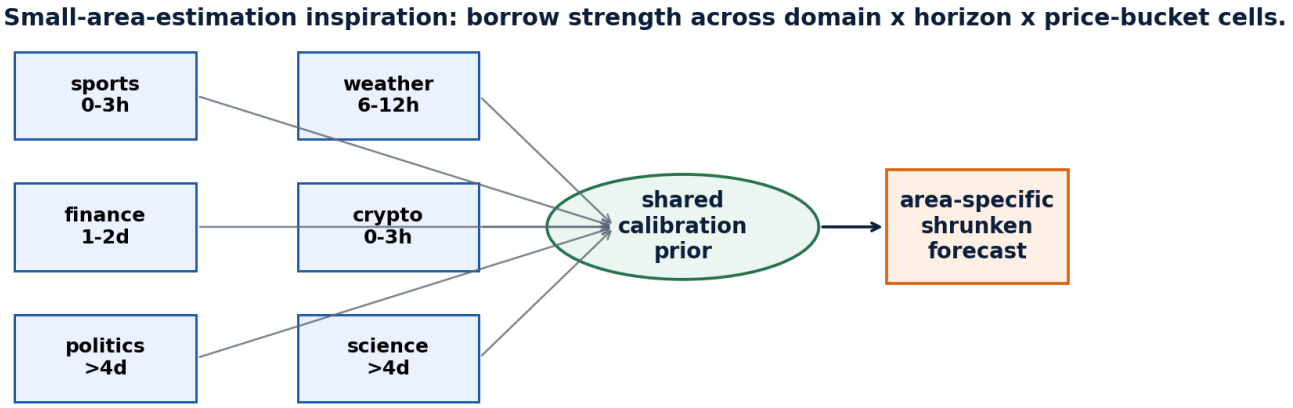

Figure 5: Small area estimation borrowing

Why this helps

We will not have enough resolved examples in every cell. For example:

```
sports, 0-3h, high-price favorite
finance, 1-2d, mid-price
politics, >4d, low-price longshot
weather, 6-12h, official-source available
```

A naive system learns each cell separately and overfits. An SAE-inspired system pools evidence.

Practical model

Start with a non-Bayesian approximation:

```
z_final = w_market * logit(p_market)
        + w_model  * logit(p_model)
        + b_domain[domain]
        + b_horizon[horizon]
        + b_price_bucket[price_bucket]
        + b_source * source_quality
        + b_spread * spread
        + b_disagreement * model_disagreement

p_final = sigmoid(z_final)
```

With limited data, regularize the coefficients hard. If using Bayesian tools later, place hierarchical priors over domain/horizon/price-bucket effects.

Credibility weighting version

A simpler version is credibility weighting:

```
p_final = market prior + credibility * (model estimate - market prior)
```

Credibility rises when:

- source quality is high;
- model ensemble agrees;

- domain has evidence that LLMs handle well;
- horizon is long enough for broad knowledge to matter;
- the market is thin or stale.

Credibility falls when:

- price is a low-probability longshot;
- spread is wide;
- source quality is weak;
- models disagree;
- the event is near resolution and market likely updates faster.

Data acquisition: APIs, scraping, and useful signals

The acquisition rule should be pragmatic:

> Use whatever signal improves forecasts, but isolate it, log it, and make sure the submitted agent can run without brittle external dependencies.

Source classes

Source class	Examples	Use
Benchmark payload	Prophet Arena events, market snapshots, outcomes	source of truth
Market APIs/pages	Kalshi, Polymarket, Manifold if relevant	external prior / microstructure reference
Official data	BLS, NOAA, sports league sources, election offices	highest-quality evidence
News/search	Reuters, AP, ESPN, Financial Times, domain outlets	current context
Social/Discord/forums	organizer announcements, community intel	operational notes, not default evidence
Scrapers	page-specific collectors	only when a valuable signal lacks a clean API

Scraper design if we use it

If a scraper is useful, build it as a research utility, not a hidden core dependency.

```
data_sources/scrapers/{source_name}.py
  -> fetch raw page
  -> parse only required fields
  -> timestamp retrieval
  -> store normalized JSON
  -> expose source_quality and staleness
  -> fail closed without breaking agent
```

Required metadata:

```
{
  "source_name": "...",
  "url": "...",
  "retrieved_at": "...",
  "published_at": "...",
  "market_id": "...",
  "fields": {...},
  "parse_confidence": 0.0,
  "used_in_forecast": true
}
```

The decision is not moral panic. The decision is expected value. If scraping gives a unique signal, build it. If it gives noisy duplicated content, skip it.

Monitoring and visualization

Monitoring is how Rob knows whether the system is improving.

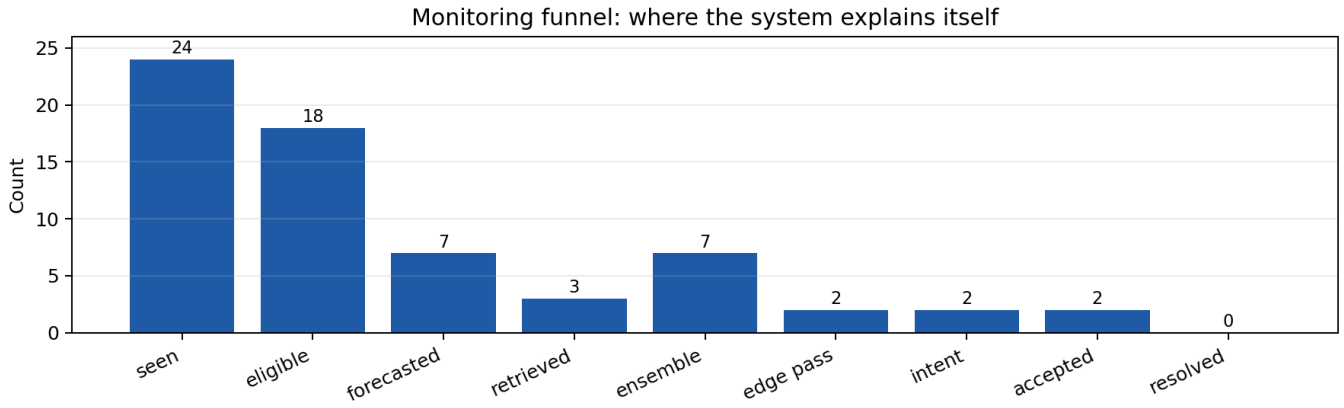

Figure 6: Monitoring funnel

Daily dashboard

Every run should produce:

```
run_id
variant
markets_seen
markets_eligible
markets_forecasted
retrieval_used
frontier_calls
opus_escalations
edge_pass_count
intents_submitted
intents_accepted
parse_failure_rate
trace_completeness
model_cost_estimate
```

Outcome dashboard

When outcomes resolve:

```
Brier overall
Brier by domain
Brier by horizon
Brier by price bucket
ECE overall
ECE by bucket
Brier skill vs market
return vs market
trade precision
skip analysis
```

Status file

`reports/status.md` should always be current:

```
# Prophet Hacks Status

Owner: Rob Sneiderman
Default variant:
Last green command:
Current blocker:
Best offline Brier:
Market-only Brier:
ECE:
Trace completeness:
Parse failure rate:
Model cost per 100 forecasts:
Open risks:
Next action:
```

If this file is clear, we are in control. If this file is vague, we are drifting.

Tests and smoke ladder

Smoke ladder

Level	Test	Pass condition
0	import/config	imports succeed, config loads
1	dry run	no API call, dirs created, config hash shown
2	toy replay	toy trace + summary + charts
3	offline forecast	predictions JSON validates
4	offline scoring	Brier/ECE/return table generated
5	connector check	external connectors fail closed
6	live test slug	tick completes, no missing logs
7	package	clean zip, one command, no secrets

Unit tests to add

```
test_edge_math.py
test_market_blend.py
test_longshot_guard.py
test_sae_shrinkage.py
test_brier_ece.py
test_source_scoring.py
test_json_schema.py
test_no_secrets.py
test_package_clean.py
test_connector_fail_closed.py
```

Paper-derived tests

Prophet Arena suggests:

- context ablation: market only vs sources only vs both vs none;
- bidirectional elicitation test;
- model conservatism vs market;
- source-quality sensitivity;
- horizon analysis;
- reasoning-to-probability audit.

Kalshi suggests:

- price-bucket diagnostics;
- longshot guard;
- spread-aware edge threshold;
- taker/executable-price penalty;
- near-close market prior weighting.

Hyperparameter search and stacking

Hyperparameters worth tuning

Component	Parameters
market blend	model weight min/max, logit shrinkage tau
longshot guard	low-price threshold, max lift without primary source
risk gate	base edge threshold, disagreement penalty, spread penalty
retrieval gate	max sources, min source quality, staleness penalty
frontier routing	near-threshold margin, disagreement threshold, hard-domain list
SAE shrinkage	pooling strength, domain/horizon/price bucket priors

Objective hierarchy

Do not optimize only PnL. Use a hierarchy:

1. Brier skill vs market.
2. ECE / reliability.
3. Risk-adjusted market return.
4. Cost per useful forecast.
5. Parse/runtime failure rate.

Time-based validation

Never random-split temporal forecasting data. Use:

```
train: older resolved events
validation: newer resolved events
holdout: newest resolved events before live
live: untouched contest evaluation
```

Toy example

Market:

```
Question: Will Team A win tonight?
YES ask = 0.62
YES bid = 0.58
Market midpoint = 0.60
```

Model forecasts:

```
GPT-5.5: 0.68
Claude Opus: 0.64
Market prior: 0.60
Source quality: 0.70
Model disagreement: 0.04
```

Blend:

```
p_final ~= 0.64
YES edge = 0.64 - 0.62 = 0.02
```

Decision: skip. The forecast leans YES, but the executable edge does not clear the threshold.

This is a good skip. We are not paid for being right in prose; we are paid for acting only when the price is wrong enough.

More realistic example: low-price longshot

Market:

```
Question: Will a low-probability political outcome happen?
YES ask = 0.08
YES bid = 0.05
Market midpoint = 0.065
```

Model output:

```
GPT-5.5: 0.16 because it finds a plausible narrative.
Claude Opus: 0.11, skeptical.
Sources: mostly commentary, no official/primary signal.
```

Naive edge:

```
YES edge = 0.16 - 0.08 = 0.08
```

Naively this reaches threshold. But the Kalshi paper warns that low-price contracts are systematically dangerous and can reflect probability overweighting. The longshot guard should require stronger evidence.

Adjusted decision:

```
source_quality low
price_bucket 0.01-0.10
model_disagreement moderate
required_edge raised to 0.12+
p_final shrunken toward market
trade skipped
```

This is exactly where an LLM-only agent loses money: it converts narrative plausibility into trade confidence.

Research artifact plan

If we do this right, the final write-up is not just "we built a bot." It is:

A monitored, market-aware, frontier-model-assisted forecasting system using hierarchical borrowing of calibration information across sparse event domains.

Potential paper/report structure:

1. Motivation: forecasting as predictive intelligence.
2. System: modular pipeline and monitored trace design.
3. Methods: market prior, frontier forecasts, retrieval, SAE-style shrinkage, longshot guard.
4. Experiments: context ablation, model routing, bidirectional elicitation, stacking.
5. Results: Brier, ECE, market return, price-bucket performance, cost.
6. Failure cases: bad sources, longshot narratives, near-resolution latency, model disagreement.
7. Lessons: what improved score and what was wasted complexity.

Immediate implementation roadmap

Patch 1: monitoring and status

```
reports/status.md
tools/monitor_run.py
tools/evaluate_offline.py
```

```
tools/check_no_secrets.py
tools/check_compliance.py
examples/toy_events.jsonl
```

Patch 2: frontier model forecast path

```
forecasting/providers_openai.py
forecasting/providers_anthropic.py
forecasting/prompts.py
forecasting/structured_output.py
forecasting/bidirectional.py
```

Patch 3: calibrated blend

```
forecasting/market_blend.py
forecasting/ensemble.py
forecasting/longshot_guard.py
forecasting/sae_shrinkage.py
tests/test_market_blend.py
tests/test_longshot_guard.py
```

Patch 4: CI and artifacts

```
.github/workflows/verify.yml
.github/workflows/offline-eval.yml
reports/runs/
```

Patch 5: optional connectors

```
data_sources/kalshi_readonly.py
data_sources/polymarket_readonly.py
data_sources/scrapers/
data_sources/README.md
config/source_policy.yaml
```

Open questions to fill in

These are not blockers, but they need final answers before submission.

Question	Current stance	Fill-in source
Are live adaptive weights allowed?	disabled until confirmed	organizer docs/Discord
Is hosted HTTP endpoint required?	local first	Prophet Arena instructions
Are external market APIs allowed as forecast context?	feature-gated, logged	contest rules
Which data source gives best offline lift?	unknown	offline eval
Does Opus review improve near-threshold decisions?	test	batch ablation
Does SAE-style shrinkage beat simple logit blend?	test	holdout eval
What is final default variant?	calibrated ensemble	pre-live eval

References and source anchors

This dossier is grounded in the following project and research sources:

- **LLM-as-a-Prophet / Prophet Arena paper:** live benchmark, market baseline, context ablation, Brier/ECE/market return, source-quality lessons, model conservatism, reasoning-to-probability alignment.
- **Makers and Takers: The Economics of the Kalshi Prediction Market:** Kalshi microstructure, favorite-longshot bias, maker/taker return pattern, low-price-contract danger, modest probability overweighting and belief disagreement.
- **Rob's baseline strategy:** simple tick loop first, trade only on explicit edge, max 5 markets per tick, max 3 trades per tick, max $100 notional, no YES/NO conflict.
- **Budget policy:** model API call caps, no open-ended loops, no frontier model on every trivial task, no unattended GPU spend, log cost per decision.
- **Operating pack:** robust reproducible agent, pinned package versions, local traces, clean run command.
- **Research spine:** scoring rules, calibration, prediction markets, online learning, retrieval credibility, forecasting-agent observability.
- **Submission checklist:** pinned env, one run command, API lifecycle, risk rules in code, JSONL traces, cost/prompt/config hashes, clean zip.

Closing direction

Build like we expect to win, but monitor like we expect every component to fail until proven otherwise.

The strongest path is not maximal complexity. It is controlled sophistication:

```
frontier model forecasts
+ market prior
+ selective evidence
+ model averaging
+ SAE-style borrowing of calibration information
+ longshot guard
+ strict edge/risk gate
+ full trace monitoring
+ clean package discipline
```

That is good enough to compete, good enough to study, and structured enough to become a serious artifact after the event.